

Foundation Models Used Acknowledgment

Group: MA_AppliedSession01_E

Unit: FIT2099 S1 2026 Assignment: Assignment 3

Foundation Models Used: ChatGPT / Claude / Gemini

Requirement 1: Quote cycle and Company Credit System

Purpose: The primary purpose of using AI was to design, refine and structure the QuotaManager class and its architectural integration into the game code. This included isolating corporate state logic like accumulated credits, company rank and deadline turn counting to enforce SRP.

AI Contribution:

We consulted the model to guide REQ1 design on implementing the quota manager to track the game turns and the company credits accumulated. The AI model helped in guiding how to fit all the tracking in the QuotaManager which fit the pre-existing system design of SuperComputer and EclipseNebula for the company credits and game turns respectively so that the SOLID principle can be carried out throughout my implementation, especially SRP. I had also consulted the AI to guide in writing the MAVEN test case for QuotaManager so that I can properly setup the mock company credits and company rank so that I can test on the boundary and negative case for the QuotaManager.

Requirement 2: Scrap Snatcher & Fleshy Tree Extension

Purpose: Implementing a resource-hoarding creature that steals depositable items, triggers loot explosion on spawn, and becomes rabid when infected. Modifying Fleshy Tree on 99-deprecated map to spawn Undead then ScrapSnatchers, evolving into a warping Monolith.

AI Contribution:

We consulted the model to guide REQ2 design decisions including: behaviour priority hierarchy (steal priority 1, wander priority 999); infection transformation logic (removing steal behaviour, adding attack behaviour); loot explosion using reflection with DEPOSITABLE_CLASSES list for extensibility (OCP); reusing UndeadFist per lecturer feedback; 33/33/33 spawn distribution using integer partitioning; creating new tree stage classes instead of modifying existing ones (OCP); Monolith teleportation via BaseTeleportStrategy reuse; and defensive null checks for edge cases.

The model also helped write comprehensive JUnit 5/Mockito test suites covering normal, edge, and boundary cases (156 tests passing), including MockedStatic for SpatialSearch and spy() for behaviour verification. All SOLID principles were followed: SRP (behaviour/action separation), OCP (extensible via interfaces), DIP (dependence on abstractions).

Requirement 3: High-Voltage Galvanic System

Purpose:

Implementing an electrical ecosystem where ChargeSource emitters release high-voltage surges that cascade through terrain, actors, and items simultaneously. The system transforms the environment dynamically, by electrifying puddles into hazards (Puddle → ElectrifiedPuddle), evolving dormant creatures into predators (DormantStaticCreature → StaticStalker), powering magnetic tools to harvest scrap remotely (Wallet with ray-cast flux blocking), and paralysing workers caught in the current (ParalyzedStatus with reflective feedback).

AI Contribution:

We consulted the model to review whether the REQ3 galvanic system design aligned with the assignment rules, including the two new abstraction requirements (ChargeSource and ChargeReactive), the six-concrete-classes distribution, and complex cross-component interactions. The AI helped guide:

- The GalvanicCharge visited-set pattern for recursion prevention between adjacent conductive tiles (PoweredFloor propagating to neighbours without infinite loops)
- Manhattan distance filtering for diamond-shaped AoE calculations in TeslaCoil.releaseCharge(), ensuring realistic electrical arc patterns rather than square explosions
- The magnetic repulsion algorithm for DormantStaticCreature, including anomaly radius accumulation (0→3), diamond-shaped pulse scanning, and item push logic with boundary checking
- The ChargeUtils.zapTile() utility design to ensure consistent surge propagation across all emitters (lightning, coils, batteries) while applying damage, status effects, metamorphosis, and inventory reactions
- I consulted the model to assist with writing comprehensive JUnit 5/Mockito test suites for the High-Voltage Galvanic System (REQ3). The AI contributed to:
 - ArgumentCaptor usage - Validating internal state of status effects (e.g., ShockedStatus duration)
 - Recursion safety verification - Testing the visited-set pattern prevents infinite loops

Requirement 4: Distorted Sanctuary System

Purpose: Implementing a spatial anomaly and protective zone system where DistortionSource grounds release corruption that warps game physics and manipulates actor behaviour, while SanctuaryTool items and the SuperComputer deploy protective fields, tactical knockback pulses, and a corporate audit protocol that integrates with the REQ1 quota system.

AI Contribution: We consulted the model to review whether the REQ4 Distorted Sanctuary design aligned with the assignment rules, including the two new abstraction requirements (DistortionSource and SanctuaryTool), the six-concrete-classes distribution, and complex cross-component interactions. The AI was consulted to verify design decisions including:

- Whether the DistortionSource interface and getGroundAs() pattern correctly satisfied DIP without instanceof, and whether the radius-3 audit scan in DistortionAuditAction was architecturally sound
- Whether the presence-aware lifecycle split between CorruptedSafeHouse and SanctuaryStatus respected SRP, and whether delegating fire spawning to spawnBlueFire() was the right decomposition
- Whether the BlueFire originalGround approach was the correct application of the Memento pattern, and whether FireStackable was appropriately scoped as a separate interface
- Whether routing KillerInstinctStatus through ContractedWorker.playTurn() was the right way to inject RageStrikeActions given the engine's constraint that Ground.tick() cannot access the ActionList
- Whether the DamageInterceptor and DistortionCapability enums were correctly scoped as dedicated enums rather than extensions of existing ones, and whether their package placement in game.sanctuary was justified

Requirement 5: Real Weather Anomaly System

Purpose:

Implementing an API-driven weather anomaly system that connects OpenWeather API data to meaningful game-world effects. The system uses dynamic API request values based on the current game state and converts weather data into anomalies such as Conductive Rain, Heat Distortion, and Storm Surge.

AI Contribution:

We consulted the model to review whether the REQ5 API feature aligned with the assignment rules, including dynamic API request construction, secure API key handling through an environment variable, and meaningful game-state impact. The model was also used to help check the structure of the API-related classes, interpreter/effect abstractions, and testing coverage for the weather anomaly system. The AI helped us refine wording for the README and feature proposal so the implementation steps, API setup, and expected behaviour were explained clearly.

Acknowledgment of Responsibility

We acknowledge that while foundation models were used to guide the implementation of these specific logic blocks, we are entirely responsible for the final code submitted. We have verified all citations and code logic for accuracy.